
This `condition and valueIfTrue or valueIfFalse` style of pseudo-ternary operator has a subtle bug: if `valueIfTrue` is a falsey value (such as `0`, `False`, `None`, or the blank string), the expression unexpectedly evaluates to `valueIfFalse` if `condition` is `True`.

But programmers continued to use this fake ternary operator anyway, and “Why doesn’t Python have a ternary operator?” became a perennial question to the Python core developers. Conditional expressions were created so programmers would stop asking for a ternary operator and wouldn’t use the bug-prone pseudo-ternary operator. But conditional expressions are also ugly enough to discourage programmers from using them. Although beautiful may be better than ugly, Python’s “ugly” ternary operator is an example of a case when practicality beats purity.

Conditional expressions aren’t exactly pythonic, but they’re not unpythonic, either. If you do use them, avoid nesting conditional expressions inside other conditional expressions:

```
>>> # Unpythonic Example
>>> age = 30
>>> ageRange = 'child' if age < 13 else 'teenager' if age >= 13 and age < 18 else
'adult'
>>> ageRange
'adult'
```

Nested conditional expressions are a good example of how a dense one-liner can be technically correct but frustrating to make sense of when reading.

Working with Variable Values

You’ll often need to check and modify the values that variables store. Python has several ways of doing this. Let’s look at a couple of examples.

Chaining Assignment and Comparison Operators

When you have to check whether a number is within a certain range, you might use the Boolean `and` operator like this: